

# Study Guide: Orders of Growth

## Instructions

This is a study guide about orders of growth with links to past lectures, assignments, and handouts, as well as additional practice problems to assist you in learning the concepts.

### Assignments

Important: For solutions to these assignments once they have been released, see [the main website](#).

- [Lab 9](#)

### Lectures

- [Efficiency](#)

### Readings

- [Section 2.8](#)

## Orders of growth

The order of growth of an algorithm is **an approximation of the time** required to run a computer program as the **input size increases**. The order of growth ignores the constant factor needed for fixed operations and focuses instead on the operations that increase proportional to input size. For example, a program with a linear order of growth generally requires double the time if the input doubles.

The order of growth is often described using either [Big-Theta](#) or [Big-O notation](#), but that notation is out of scope for this course.

This table summarizes the most common orders of growth:

| Order       | Big-Theta       | Example                            |
|-------------|-----------------|------------------------------------|
| Constant    | $\Theta(1)$     | Indexing an item in a list         |
| Logarithmic | $\Theta(\lg N)$ | <b>Repeatedly halving a number</b> |

|             |               |                                        |
|-------------|---------------|----------------------------------------|
| Linear      | $\Theta(n)$   | Summing a list                         |
| Quadratic   | $\Theta(n^2)$ | Summing each pair of numbers in a list |
| Exponential | $\Theta(2^n)$ | Visiting each node in a binary tree    |

## Constant time

When an algorithm has a constant order of growth, it means that it always takes a fixed number of steps, no matter how large the input size increases.

As an example, consider accessing the first element of a list:

```
first_post = posts[0]
```

Even if the list grows to be a million items long, that operation will always require a single step.

We can visualize that relationship as a table:

| List size | Steps |
|-----------|-------|
| 1         | 1     |
| 10        | 1     |
| 100       | 1     |
| 1000      | 1     |

We can also visualize it as a graph:

 Constant graph

A constant run time is ideal, but is typically not possible for algorithms that process multiple pieces of data.

## Logarithmic time

When an algorithm has a logarithmic order of growth, it increases proportionally to the [logarithm](#) of the input size.

The [binary search algorithm](#) is an example of an algorithm that runs in logarithmic time.

Here's the pseudocode:

```
def search_list(nums, target_num):
    """ Returns the index of TARGET_NUM in sorted list NUMS or -1 if not found.
    >>> search_list([1, 2, 3, 4], 3)
    2
    >>> search_list([14, 23, 37, 48, 59], 23)
    1
    >>> search_list([14, 23, 37, 48, 59], 47)
    -1
    """
    min_index = 1
    max_index = len(nums)
    while min_index <= max_index:
        middle_index = (min_index + max_index) // 2
        if target_num == nums[middle_index]:
            return middle_index
        elif target_num > nums[middle_index]:
            min_index = middle_index + 1
        else:
            max_index = middle_index - 1
    return -1
```

The algorithm uses a loop to look at multiple items in the list, but crucially, it does not look at *every* item in the list. Specifically, it looks at  $\lg_2(n)$  items, where  $n$  is the number of items in the list.

We can visualize that relationship in a table:

| List size | Steps |
|-----------|-------|
| 1         | 1     |
| 10        | 4     |
| 100       | 7     |
| 1000      | 10    |

We can also see that as a graph:

 Logarithmic graph

The number of steps is definitely increasing as input size increases, but at a very slow rate.

## Linear time

When an algorithm has a linear order of growth, its number of steps increases in **direct proportion** to the input size.

The aptly-named linear search algorithm runs in linear time.

The code shows its simplicity compared to binary search:

```
def search_list(nums, target_num):
    """ Returns the index of TARGET_NUM in an unsorted list NUMS or -1 if not found.
    >>> search_list([3, 2, 1, 4], 3)
    2
    >>> search_list([14, 59, 99, 23, 37, 22], 23)
    3
    >>> search_list([14, 59, 99, 23, 37, 22], 47)
    -1
    """
    index = 1
    while index < len(nums):
        if nums[index] == target_num:
            return index
        index += 1
    return -1
```

This time, the loop looks at *every* item in the list. This exhaustive search is necessary to search for items in an unsorted list, since there's no way to narrow down where a particular item might be. This algorithm will always require at least as many steps as items in the list.

We can see that in table form:

| List size | Steps |
|-----------|-------|
| 1         | 1     |
| 10        | 10    |
| 100       | 100   |
| 1000      | 1000  |

Or as a graph:

 Linear graph

## Quadratic time

When an algorithm has a quadratic order of growth, its steps increase in proportion to the input size squared.

Several **list sorting algorithms run in quadratic time**, like selection sort. That algorithm starts from the front of the list, then keeps finding the next smallest value in the list and swapping it with the current value.

Here's pseudocode for selection sort:

```
def linear_sort(nums):
    """Performs an in-place sorting of NUMS.
    >>> l = [2, 3, 1, 4]
    >>> linear_sort(l)
    >>> l
    [1, 2, 3, 4]
    """
    i = 0
    while i < len(nums):
        min_index = i
        j = i + 1
        # Find next smallest value
        while j < len(nums):
            if nums[j] < nums[min_index]:
                min_index = j
            j += 1
        # Swap if new minimum found
        if min_index != i:
            nums[i], nums[min_index] = nums[min_index], nums[i]
        i += 1
```

The important thing to notice about the algorithm is the nested loop: it loops through each items in the list, and loops *again* through the remaining items for each of those items. In this case, the algorithm ends up looking at  $\frac{1}{2} * (n^2 - n)$  values, where  $n$  is the number of items in the list.

This table shows how many items it'd examine for lists of increasing sizes:

| List size | Steps  |
|-----------|--------|
| 1         | 1      |
| 10        | 45     |
| 100       | 4950   |
| 1000      | 499500 |

Here's what that looks like in graph form:

 Quadratic graph

Both the table and the graph show that the number of steps for this algorithm increases at a *much* faster rate than the previous ones.

# Exponential time

When an algorithm has a superpolynomial order of growth, its number of steps increases faster than a polynomial function of the input size.

An algorithm often requires superpolynomial time when it must look at every permutation of values. For example, consider an algorithm that generates all possible numerical passwords for a given password length.

For a password length of 2, it generates 100 passwords:

```
00 01 02 03 04 05 06 07 08 09
10 11 12 13 14 15 16 17 18 19
20 21 22 23 24 25 26 27 28 29
30 31 32 33 34 35 36 37 38 39
40 41 42 43 44 45 46 47 48 49
50 51 52 53 54 55 56 57 58 59
60 61 62 63 64 65 66 67 68 69
70 71 72 73 74 75 76 77 78 79
80 81 82 83 84 85 86 87 88 89
90 91 92 93 94 95 96 97 98 99
```

That algorithm requires at least  $10^2$  steps, since there are 10 possibilities for each digit (0-9) and it must try out every possibility for each of the 2 digits.

For any given password length of  $n$ , the algorithm requires  $10^n$  steps. That run time increases incredibly quickly, since each additional digit requires 10 times the number of steps.

This table shows how fast that grows for just the first 5 digits:

| Digits | Steps  |
|--------|--------|
| 1      | 10     |
| 2      | 100    |
| 3      | 1000   |
| 4      | 10000  |
| 5      | 100000 |

Here's what that looks like as a graph:

 Exponential graph

## All together now

Now that we've seen examples of possible run times for algorithms, let's compare them on a graph:

 All graphs overlaid

That graph makes it even more clear that there's a definite difference in these run times, especially as input size increases. Since modern computer programs increasingly deal with large data sets (like from millions of users or sensors), the run time efficiency matters quite a bit.

## Practice Problems

---

### Q1: Is Prime

What is the order of growth of `is_prime` in terms of `n`?

```
def is_prime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Choose one of:

- Constant
- Logarithmic
- Linear
- Quadratic
- Exponential
- None of these

[Toggle Solution](#)

### Q2: Growth: Bar

What is the order of growth of `bar` in terms of `n`?

```
def bar(n):
    i, sum = 1, 0
    while i <= n:
        sum += biz(n)
        i += 1
    return sum

def biz(n):
    i, sum = 1, 0
    while i <= n:
        sum += i**3
        i += 1
    return sum
```

Choose one of:

- Constant
- Logarithmic
- Linear
- Quadratic
- Exponential
- None of these

[Toggle Solution](#)

What is the order of growth of `foo` in terms of `n`, where `n` is the

length of `lst`? Assume that slicing a list and calling `len` on a list can both be done in constant time.

```
def foo(lst, i):
    mid = len(lst) // 2
    if mid == 0:
        return lst
    elif i > 0:
        return foo(lst[mid:], -1)
    else:
        return foo(lst[:mid], 1)
```

Logarithmic  $\Theta(\log(n))$ .

*Explanation:* A single recursive call is made in the body of `foo` on half the input list (either the first half or the second half depending on the input flag `i`). The base case is executed when the list either is empty or has only one element. We start with an  $n$  element list and halve the list until there is at most 1 element, which means there will be  $\log(n)$  total calls. Each call, constant work is done if we ignore the recursive call. The total runtime is then  $\log(n) * \theta(1)$ .

*Note:* We simplified this problem by assuming that slicing a list takes constant time. In reality, this operation is a bit more nuanced and may take linear time. As an additional exercise, try

determining the order of growth of this function if we assuming slicing takes linear time.

### Q3: Bonk

Describe the order of growth of the function below.

```
def bonk(n):  
    sum = 0  
    while n >= 2:  
        sum += n  
        n = n / 2  
    return sum
```

Choose one of:

- Constant
- Logarithmic
- Linear
- Quadratic
- Exponential
- None of these

[Toggle Solution](#)